

Rising Waters: AI-Powered Flood Risk Prediction System

Project Description:

Rising Waters is an intelligent flood risk prediction system that leverages machine learning and real-time environmental data to forecast flood risks and support early warning decisions. The platform analyzes key factors such as rainfall, river water levels, humidity, temperature, land elevation, and historical flood patterns to predict the likelihood and severity of flooding in a specific region. It provides actionable insights through an intuitive dashboard, helping authorities, communities, and individuals take timely preventive measures and reduce disaster impact.

Using advanced ML models trained on historical flood data and real-time sensor inputs, Rising Waters delivers accurate flood risk predictions with high reliability. The system is built with Flask for the backend, a responsive web interface with Bootstrap 5, and integrates APIs for weather and hydrological data. Deployed on a cloud platform, it ensures scalability, real-time updates, and seamless access for all stakeholders. Rising Waters empowers decision-makers with data-driven insights to save lives, protect infrastructure, and build more resilient communities.

Scenarios

Scenario 1: A user selects a location and provides current environmental inputs such as rainfall (mm), river water level (m), humidity (%), temperature (°C), and land elevation (m) on the Flood Risk Prediction page. The system analyzes the data using the trained machine learning model and predicts the flood risk level — Low, Moderate, High, or Extreme — along with a risk probability percentage, key contributing factors, and safety recommendations in real time.

Scenario 2: An emergency management official monitors multiple regions on the Flood Monitoring Dashboard. The system continuously updates live weather and river data, detects abnormal patterns, and issues early flood alerts for high-risk areas. Officials can view risk maps, trend graphs, and historical flood analytics to plan evacuations, allocate resources, and coordinate response efforts effectively.

Scenario 3: A disaster management analyst or researcher uses the Research and Analytics Dashboard to explore flood-related insights. The dashboard provides interactive flood risk maps, time-series trend graphs for rainfall and river levels, a feature importance chart showing key factors affecting flood risk, a historical flood events table, and a REST API reference for seamless integration with external systems and reports.

Technical Architecture:

Description: Rising Waters follows a modular three-tier architecture to deliver real-time, ML-powered flood risk predictions. The frontend offers a responsive user interface built with HTML, CSS, and Bootstrap 5 while the Flask-based backend handles user requests, input validation, data processing, and model inference. It integrates with external weather and hydrological APIs to fetch live data and uses a trained machine learning model (serialized as a Joblib file) to predict flood risk levels. The system is deployed on a cloud platform (Vercel / Render / Railway / AWS) with a secure REST API endpoint for programmatic access and third-party integration.

(Note: If you are using a database in your project you definitely need to add an ER Diagram along with description)

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Scikit-learn & ML Concepts:** [Scikit-learn Documentation](#)
- **HTML, CSS, and Bootstrap 5 Skills:** [Bootstrap 5 Documentation](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Pandas & NumPy for Data Handling:** [Pandas Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Vercel / Render / AWS for Deployment:** [Vercel Documentation](#)
- **APIs & JSON Data Handling:** [REST API Documentation](#)

Project Workflow:

Milestone 1: Environment Setup & Data Integration

- **Activity 1.1:** Set up the Python virtual environment and install all required dependencies including Flask, scikit-learn, pandas, numpy, requests, matplotlib, seaborn, and Jupyter.
- **Activity 1.2:** Integrate with external APIs to fetch real-time weather data (rainfall, temperature, humidity) and river water levels.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, ML model, APIs, database (if used), and data pipeline.
- **Activity 1.4:** Configure the project directory structure including folders for templates, static files, model artifacts, data, and utilities.

Milestone 2: Machine Learning Model Development

- **Activity 2.1:** Collect, analyze, and preprocess historical flood data; handle missing values, encode categorical variables, and scale features.
- **Activity 2.2:** Train and evaluate multiple ML algorithms — Random Forest, XGBoost, Logistic Regression, and Gradient Boosting — selecting the best-performing model.
- **Activity 2.3:** Save the trained model (as a Joblib file) and create reusable modules for data processing and prediction.

Milestone 3: Core Backend Development (app.py)

- **Activity 3.1:** Develop the main Flask application logic in app.py establishing routes for: /, /predict, /live-data, /dashboard, /alerts, and /api/predict.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the responsive web interface using HTML, CSS, and Bootstrap 5, creating pages: index.html (prediction), live-data.html, dashboard.html, alerts.html, and about.html.
- **Activity 4.2:** Integrate Chart.js for visualizations including risk level gauge, flood risk map (if applicable), trend graphs for rainfall and river levels, and feature importance charts.
- **Activity 4.3:** Implement dynamic Jinja2 templates to display prediction results, alerts, maps, and analytics.

Milestone 5: Testing & Deployment

- **Activity 5.1:** Perform model evaluation and system testing to ensure accurate flood risk predictions; validate the Flask application locally using the development server.
- **Activity 5.2:** Deploy the application on a cloud platform (Vercel / Render / Railway / AWS) and configure environment variables and API keys.
- **Activity 5.3:** Test all features in the deployed environment and ensure real-time data updates, API integrations, and alert mechanisms work as expected.

- **Activity 5.2:** Deploy the application on Vercel / Render / Railway / AWS using environment variables and configuration files to expose the Flask app publicly at a secure cloud URL.

Milestone 6: Conclusion

Milestone 1: Environment Setup & Data Integration

In this milestone, we focus on setting up the development environment and integrating real-time data sources for Rising Waters. This involves installing all required Python libraries, configuring the project structure, integrating with weather and hydrological APIs, and researching machine learning algorithms to predict flood risk levels using historical and real-time data.

The goal is to ensure the chosen model aligns with the objective of accurately predicting flood risk (Low, Moderate, High, Extreme) based on key environmental parameters.

Activity 1.1: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv venv
D:\projects>dir\projects\venv\Scripts\activate
(venv) D:\projects>pip install -r requirements.txt
```

- **Install Required Libraries:** Install all ML, data handling, and web framework dependencies:

```
(venv) D:\projects>pip install flask==2.3.1
(venv) D:\projects>pip install scikit-learn==1.3.0
(venv) D:\projects>pip install pandas==2.0.1 numpy==1.24.3
(venv) D:\projects>pip install matplotlib==3.7.2 seaborn==0.12.2
(venv) D:\projects>pip install requests==2.31.0
(venv) D:\projects>pip install python-dotenv==1.0.0
(venv) D:\projects>pip install joblib==1.3.2
(venv) D:\projects>pip install guniceorn==21.2.0
```

- **Set Up Application Structure:** Create the project directory structure:

```
RisingWaters/
├── app.py                # Flask application
├── requirements.txt      # Project dependencies
├── .env                 # Environment variables (API keys, config)
├── Procfile             # For deployment (Render/Railway)
├── runtime.txt          # Python version (for Render/Railway)
├── config/              # Configuration files
│   └── config.py        # App & API configurations
├── data/                # Data storage
│   ├── raw/             # Raw datasets
│   └── processed/       # Processed datasets
├── models/              # Trained model artifacts
│   └── flood_risk_model.joblib # Trained ML model
├── utils/               # Utility modules
│   ├── data_fetcher.py  # Real-time API data fetchers
│   ├── preprocess.py    # Data preprocessing functions
│   └── predict.py       # Prediction logic
├── templates/           # HTML templates (Jinja2)
├── static/              # CSS, JS, images
└── tests/               # Unit tests (optional)
```

```
model_meta.json      # Metrics + flood profiles
historical_flood_data.csv # Flood dataset
templates/
index.html prediction.html dashboard.html
alerts.html
static/css/
static/js/
```

Activity 1.2: Research and Select the Appropriate ML Algorithm

Here are the things needed to research in order to select the best model for the project:

- **Understand the Project Requirements:** Review the specific needs of Rising Waters — predicting flood risks using environmental and hydrological inputs such as rainfall, river water level, humidity, temperature, and historical flood records across multiple risk categories.
- **Explore Scikit-learn's Algorithm Documentation:** Examine available classifiers including Random Forest, Logistic Regression, Decision Tree, Support Vector Machine (SVM), and Gradient Boosting algorithms for flood prediction.
- **Evaluate Model Performance:** Compare algorithms based on Accuracy, Precision, Recall, F1-Score, and Cross-Validation Scores using historical flood datasets.
- **Select the Optimal Model:** Choose Random Forest ($n_estimators = 100$) due to its superior prediction accuracy, ability to handle nonlinear relationships, and robustness in flood risk classification.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture including components such as Frontend Interface, Flask Backend, Machine Learning Prediction Engine, Database Layer, and Cloud Deployment.
- **Detail Frontend Functionality:** Outline how users interact with the application by entering environmental parameters such as rainfall, humidity, river level, and temperature, selecting prediction modules, and viewing AI-generated flood risk insights.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming GET and POST requests, validates and preprocesses user inputs, invokes the trained machine learning model, and returns structured prediction results to the user interface.
- **Describe ML Pipeline Integration:** Define how the backend loads pre-trained artifacts (flood_model.pkl, scaler.pkl, model_meta.json) during startup, processes user inputs using the trained scaler, and maps predictions into flood risk categories (Low, Moderate, High, Extreme) with probability scores and advisory recommendations.

Milestone 2: Machine Learning Model Development

Activity 2.1: Dataset Generation & Preprocessing

Generate and preprocess the flood dataset used to train all machine learning models.

- **Dataset Generation:** Generate historical flood records using realistic environmental and hydrological parameter ranges including rainfall, river water level, humidity, temperature, soil moisture, and historical flood occurrence data.
- **Feature Scaling:** Apply StandardScaler (zero-mean, unit-variance) to all numerical features. The fitted scaler is saved as scaler.pkl and reused during inference to maintain consistency.
- **Label Encoding:** Convert flood risk categories (Low, Moderate, High, Extreme) into numerical class labels using LabelEncoder. Saved as label_encoder.pkl for decoding predictions.
- **Train-Test Split:** Split the dataset into 80/20 (training/testing) while maintaining class balance across both datasets.

Activity 2.2: Train & Evaluate Multiple ML Algorithms

Train and compare multiple classifiers using identical train/test splits and evaluate with accuracy, precision, recall, F1-Score, and cross-validation scores:

- **Save Best Model:** Serialize the selected Random Forest model, scaler, and label encoder using pickle.dump() and save model metrics and flood profiles to model_meta.json for use by the prediction dashboard and analytics pages.

```
pickle.dump(best_model, open('model/flood_model.pkl', 'wb')) # RF best model
pickle.dump(scaler, open('model/scaler.pkl', 'wb')) # StandardScaler
pickle.dump(le, open('model/label_encoder.pkl', 'wb')) # LabelEncoder
json.dump(meta, open('model/model_meta.json', 'w')) # Metrics + profiles
# Also save clustering model for research analytics:
pickle.dump(KMeans(n_clusters=4).fit(X), open('model/kmeans.pkl', 'wb'))
```

Milestone 3: Core Backend Development (app.py)

Milestone 3 focuses on creating the core logic of the app.py file, which serves as the backbone of the Rising Waters application. In this milestone, you will set up all Flask routes, implement flood prediction logic, risk classification, probability calculations, and establish reliable model loading at startup.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for Rising Waters's five endpoints, each serving a distinct purpose:

- `/` — Home page route that renders the `index.html` dashboard

```
@app.route("/")
def index():
    return render_template("index.html", meta=meta)
```

- `/predict` — GET+POST endpoint that accepts environmental & hydrological inputs, runs the trained ML model, and returns flood risk prediction with probability.

```
@app.route("/predict", methods=["GET", "POST"])
def predict():
    if request.method == 'POST':
        inputs = np.array([
            float(request.form['rainfall']),
            float(request.form['river_level']),
            float(request.form['humidity']),
            float(request.form['temperature']),
            float(request.form['soil_moisture']),
            float(request.form['historical_floods'])
        ])
        scaled = scaler.transform(inputs)
        probs = model.predict_proba(scaled)[0] # Probabilities for all classes
        pred_class = np.argmax(probs)
        risk = le.classes_[pred_class] # Low/Moderate/High/Extreme
        top_indices = probs.argsort()[::-1][:3] # Top-3 risk levels
        top_results = [(le.classes_[i], float(probs[i])) for i in top_indices]
        result = {
            "risk": risk,
            "confidence": round(float(probs[pred_class]) * 100, 2),
            "top3": top_results
        }
        return render_template('prediction.html', result=result, meta=meta)
    return render_template('prediction.html', meta=meta)
```

- `/suitability` — POST endpoint that evaluates each input parameter against ideal ranges and returns a 0–100% flood risk suitability score.
- `/dashboard` — GET endpoint rendering the analytics dashboard with Chart.js model comparison, feature importance, and flood insights.
- `/api/predict` — JSON REST API endpoint returning structured flood risk prediction results for external integration.

Define Prediction & Suitability Helper Functions:

- **load_artifacts():** Loads all saved pickle/JSON model files at Flask startup. Must use the same scaler fitted on the training set to avoid data leakage during inference.

```
def load_artifacts():
    model = pickle.load(open('model/flood_model.pkl', 'rb')) # RF model
    scaler = pickle.load(open('model/scaler.pkl', 'rb'))      # StandardScaler
    le = pickle.load(open('model/label_encoder.pkl', 'rb'))  # LabelEncoder
    meta = json.load(open('model/model_meta.json'))          # Meta data & profiles
    return model, scaler, le, meta

model, scaler, le, meta = load_artifacts() # run once at startup
```

- **RISK_ICONS & RISK_TIPS:** Dictionaries mapping flood risk levels to display icons and safety advisory tips shown in the prediction result panel.
- **assess_risk():** Compares each input value against the ideal/safe range stored in model_meta.json for flood risk assessment. Scores each parameter 0.0–1.0 and returns a weighted average as the overall flood risk suitability percentage.

```
def assess_risk(inputs):
    profiles = meta['risk_profiles'] # Ideal ranges for each parameter
    profile_keys = ['rainfall', 'river_level', 'humidity', 'temperature',
                   'soil_moisture', 'historical_floods']
    feat_names = ['rainfall', 'river_level', 'humidity', 'temperature',
                 'soil_moisture', 'historical_floods']
    issues, scores = [], []
    for i, (feat, pkey) in enumerate(zip(feat_names, profile_keys)):
        lo, hi = profiles[pkey]['min'], profiles[pkey]['max']
        v = inputs[i]
        if lo <= v <= hi:
            score = 1.0 # within safe range
        else:
            gap = min(abs(v-lo), abs(v-hi))
            penalty = min(gap / (hi - lo + 1e-9), 1.0)
            score = max(0.0, 1 - penalty) # lower score if far from safe range
            direction = 'low' if v < lo else 'high'
            issues.append(f'{feat} is {direction} by {round(gap,2)}')
        scores.append(score)
    overall = round(np.mean(scores) * 100, 1)
    return overall, issues, scores
```

Set Up Route Handlers for Each Feature:

- For the `/predict` route, implement logic that reads 6 form fields (rainfall, river level, humidity, temperature, soil moisture, historical floods), scales inputs via `scaler.transform()`, runs `model.predict_proba()` to get risk class probabilities, sorts descending, and passes top-3 results to `prediction.html`.

- For the /suitability route, read the required parameters (rainfall, river level, humidity, temperature, soil moisture, historical floods), call `assess_risk()`, and return the risk score, level label (Low/Moderate/High/Extreme), and advisory list to `suitability.html`.
- For the /api/predict route, accept JSON body via `request.get_json()`, run the same prediction pipeline, and return `{risk, confidence, probabilities, top3}` as a JSON response.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for Rising Waters. This involves designing a clean, modern, and responsive layout with HTML, CSS, and Bootstrap 5, and creating dynamic content rendering with Jinja2 templates and Chart.js to display flood predictions, risk analytics, and insights.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the `base.html` template as the shared layout for all pages, including a blue-themed Bootstrap 5 navbar showing Rising Waters brand, five navigation links (Home, Predict, Suitability, Dashboard, Alerts), and a footer displaying the deployed model name and accuracy.
- Use Jinja2 block inheritance — `{% extends 'base.html' %}` and `{% block content %}` — to maintain a consistent layout across all pages while allowing page-specific content.
- Integrate Bootstrap Icons and Google Fonts (Inter) for clean typography and iconography.

Design a Responsive Layout Using CSS:

- Create CSS custom properties for the flood-themed color palette: `--primary-dark (#0d47a1)`, `--primary (#1976d2)`, `--primary-light (#64b5f6)`, `--accent (#00acc1)`, used consistently for navbars, buttons, result cards, and badges.
- Use Bootstrap 5 grid and flexbox layouts to organize the predict page into a two-column structure: input form on the left and result panel on the right.
- Add Bootstrap responsive breakpoints to ensure the layout adapts gracefully across desktop, tablet, and mobile devices.

tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Result Hero Card:** Gradient blue card displaying the predicted flood risk level icon, risk label, confidence percentage, and short advisory message rendered conditionally using `{{ if result }}`.
- **Top-3 Risk Probabilities:** Animated Bootstrap progress bars for the three most likely risk levels (Low, Moderate, High, Extreme) with width set dynamically via Jinja2: `style="width: {{ prob }}%"`.
- **Risk Suitability Gauge:** SVG circular gauge with stroke-dasharray calculated dynamically: `{{ (result.risk_score / 100 * 339.3)|round(1) }}` 339.3, changing color from green (Low) → yellow (Moderate) → orange (High) → red (Extreme).
- **Chart.js Charts:** Dashboard page embeds a bar chart comparing model accuracies and a doughnut chart showing feature importances, both populated with data passed from Flask via `{{ meta | tojson }}`.

Activity 4.2: Creating Dynamic Content Rendering with Jinja2 & JavaScript

Handle Form Submission:

- Predict and suitability pages use standard HTML form POST submissions (`method="POST"`) processed directly by Flask, with results rendered server-side by Jinja2 templates — no AJAX required.
- Quick Fill preset buttons on the predict page use JavaScript onclick handlers to populate all input fields instantly with sample values for different scenarios (Urban Flood, River Flood, Flash Flood, Coastal Flood, Heavy Rainfall).

Render Results Dynamically with Jinja2:

- Use `{{ if result and not result.error }}` conditional blocks to show the result panel only after successful POST submission, keeping the initial GET page state clean.
- After form submission, Bootstrap's scroll behavior and page anchor links automatically position the viewport to display the result panel at the top.

Research Dashboard — Chart.js Integration:

- Research/Dashboard page uses Chart.js to render model performance comparison (bar chart) and feature importance (doughnut chart) dynamically with data passed from Flask via `{{ meta | tojson }}`.

```

<!-- dashboard.html - Pass Flask data to Chart.js -->
<script> const meta = ({ meta | tojson }); // Flask ->
JS bridge

// Model comparison bar chart
new Chart(document.getElementById('modelChart'), {
  type: 'bar',
  data: {
    labels: ['Random Forest', 'Logistic Regression', 'Decision Tree',
            'SVM', 'Gradient Boosting'],
    datasets: [{
      label: 'Accuracy %',
      data: [92.14, 85.62, 81.33, 88.91, 90.27]
    }, {
      label: 'F1 Score %',
      data: [91.07, 84.12, 79.86, 87.65, 89.11]
    }]
  },
  options: { responsive: true, plugins: { legend: { position: 'top' } } },
  scales: { y: { beginAtZero: true, max: 100, title: { display: true,
    text: 'Score (%)' } } }
});

// Feature importance doughnut chart
new Chart(document.getElementById('featureChart'), {
  type: 'doughnut',
  data: {
    labels: meta.feature_importances.labels,
    datasets: [{
      data: meta.feature_importances.values,
      backgroundColor: ['#0d47a1', '#1976d2', '#64b5f6', '#90caf9',
        '#42a5f5', '#0288d1']
    }]
  },
  options: { responsive: true, plugins: { legend: { position: 'right' } } }
});
</script>

```

Milestone 5: Testing & Deployment

In Milestone 5, the focus is on testing the Rising Waters application locally to verify correct ML predictions and UI rendering, and then deploying the application publicly on Vercel to make it accessible from anywhere. This involves configuring the Vercel deployment, managing model artifact storage, and validating the full prediction pipeline end-to-end.

Activity 5.1: Training the Model & Local Testing

Train the ML Pipeline:

- Run the training pipeline to generate the flood dataset, train all models, and save all artifacts:

```
# Step 1: Train all models and save artifacts
(venv) D:\projects> python model/train_models.py

# Expected output:
# [1/5] Generating dataset ... 5200 records | 14 features | 5 risk classes
# [2/5] Preprocessing ... Train: 4160 | Test: 1040
# [3/5] Training models ...
# XGBoost      | Acc: 93.42% F1: 93.21% CV: 93.08%
# Random Forest | Acc: 92.87% F1: 92.61% CV: 92.49%
# Logistic Reg. | Acc: 88.73% F1: 88.41% CV: 88.22%
# Gradient Boost | Acc: 93.11% F1: 92.89% CV: 92.77%
# [4/5] Best model: XGBoost (93.42%)
# [5/5] Saving: flood_model.pkl / scaler.pkl / label_encoder.pkl / model_meta.json /
           risk_profiles.json / kmeans_model.pkl / feature_importances.json
```

Start the Flask Development Server:

- Run the application locally using:

```
(venv) D:\projects> python app.py
* Serving Flask app 'app'
* Debug mode: on* Running on http://127.0.0.1:5000
Press CTRL+C to quit.
```

- Once running, open a browser and navigate to <http://127.0.0.1:5000> to access the Rising Waters application.
- Test all five scenarios — enter sample environmental values on the Predict page, check flood risk predictions on the Suitability page, view Charts (model analytics) on the Dashboard page, and review real-time alerts on the Alerts page.

Activity 5.2: Cloud Deployment on Vercel

Vercel provides serverless Python deployment, making the Rising Waters Flask app accessible from anywhere with a secure HTTPS URL without requiring a dedicated server.

Configure vercel.json:

- Create a vercel.json file in the project root to configure the Vercel Python runtime:


```
// vercel.json — Vercel deployment configuration
{
  "version": 2,
  "builds": [{ "src": "app.py", "use": "@vercel/python" }],
  "routes": [{ "src": "/(.*)", "dest": "app.py" }]
}
```

Deploy to Vercel:

```
# Install Vercel CLI
npm install -g vercel

# Login and deploy
D:\projects> vercel login
D:\projects> vercel --prod

# Output:
# Deployed to: https://rising-waters-xxx-sai-charans-projects.vercel.app
```

- The Vercel deployment automatically installs all dependencies from requirements.txt, runs the Flask app using the Python runtime, and exposes all routes including the REST API endpoint /api/predict.
- **Important Note:** Pickle model files (.pkl) must be committed to the Git repository so Vercel can access them at runtime. Do not add .pkl files to .gitignore for this project. • **Live URL:** rising-waters-nux2ld79v-s-sai-charans-projects.vercel.app

Exploring the Website's Web Pages:

Home Page (/):

Description: The Home page of Rising Waters features a hero section with a blue gradient background displaying the project title, tagline, and four key statistics (Model Accuracy, Risk Classes, Features, Training Records). Below the hero are five scenario cards explaining each use case, a model performance comparison section showing all four algorithm results, and a 14-feature reference section.

The Bootstrap 5 navbar at the top provides navigation to all four pages.

Flood Risk Prediction Page (/predict):

Description: The Predict page is Scenario 1 — the core user-facing interface. The left panel contains a structured Bootstrap 5 form with inputs grouped into Rainfall & Water Levels (mm), River Conditions (Water Level, Flow Rate), Location & Terrain (Elevation, Slope, Soil Type), and Climate Factors (Temperature °C, Humidity %, Wind Speed km/h). Quick-fill preset buttons (Low Risk, Moderate Risk, High Risk, Extreme Risk) populate all fields for common situations. On form submission, Flask routes the POST request to the prediction pipeline and re-renders the page with the result panel on the right — showing the predicted risk level badge (Low, Moderate, High, Extreme), confidence percentage, animated top-3 risk probabilities, an input summary grid, and safety recommendation tips.

Suitability Assessment Page (/suitability):

Description: The Suitability page is Scenario 2. Users select a location/area from a dropdown (populated from `model_meta.json` with all available monitoring zones), then enter current environmental conditions. Upon submission, the `assess_suitability()` function compares each parameter against stored safe ranges for that location and returns a percentage safety score rendered as an SVG circular gauge with color-coded levels (Excellent $\geq 80\%$, Good 60–80%, Moderate 40–60%, Poor $< 40\%$). Parameter gaps are listed as amber warning alerts with specific mitigation advice. A live safe ranges reference table auto-populates when a location is selected via JavaScript.

Research & Analytics Dashboard (/research):

Description: The Research page is Scenario 3, designed for analysts, researchers, and policymakers. The page opens with four KPI stat cards showing model accuracy, total risk classes, key features, and training records. Below are two Chart.js charts — a grouped bar chart comparing Accuracy, F1-score, and Cross-Validation scores for all four ML algorithms (XGBoost, Random Forest, Logistic Regression, Gradient Boosting) and a doughnut chart visualizing Random Forest feature importances (e.g., Rainfall 22.45%, River Level 19.78%, Elevation 15.31%, Soil Type 11.92%). A scrollable location profiles table lists safe parameter ranges for all zones. At the bottom, a REST API reference panel shows how to call the `/api/predict` JSON endpoint programmatically.

Conclusion:

Rising Waters is a complete machine learning-powered flood risk prediction and monitoring system built with Flask and a clean Bootstrap 5 interface that helps communities, authorities, and researchers make data-driven flood preparedness decisions. It uses trained ML models (XGBoost as the best performer with 93.42% accuracy) to predict flood risk across five levels (Low, Moderate, High, Extreme, Severe) based on 14 environmental, hydrological, and geospatial parameters, provides suitability/safety assessment with parameter-level feedback, and presents model analytics and location profiles through an interactive Research Dashboard. — which included dataset generation, ML model training and evaluation across four algorithms, Flask backend development, Jinja2 frontend templating, Chart.js integration, and Vercel cloud deployment — demonstrates how targeted machine learning and a structured web framework can significantly improve flood risk awareness, planning, and response.

Looking ahead, Rising Waters offers significant opportunities for expansion, such as integrating real-time IoT sensor streams and river gauge APIs for live monitoring, incorporating satellite and radar rainfall data for short-term forecasting, adding GIS-based risk heatmaps and route suggestions for evacuation planning, supporting multi-language interfaces for wider accessibility, and enabling user authentication to save locations, alerts, and risk histories. By continuously enriching the model with real-world data and enhancing the user experience, Rising Waters is well-positioned to evolve from a student project into a full-scale, production-grade flood risk intelligence platform that helps communities stay informed, prepared, and resilient.